Clase 13 · Sistema operativo, llamadas a sistema y procesos

Fundamentos de Sistemas Embebidos · M.C. José Mauricio Matamoros de María y Campos · lunes 12 oct 2026 · Tema 3 (3.1–3.2) · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Primera clase de la Unidad 3 (segundo parcial). Lo marcado (material) viene de las diapositivas.

3.1.1 Arquitectura de Windows frente a Linux

Windows, de arriba abajo: aplicaciones y subsistemas → **capa ejecutiva** (manejador de E/S, monitor de referencias de seguridad, LPC, memoria virtual, objetos, Plug and Play, energía, ventanas/GDI) → microkernel, y en paralelo los drivers de kernel y la **HAL** (Hardware Abstraction Layer).

- La HAL es "una muleta para los tiempos antediluvianos": un diccionario (Torre de Babel) para que hardware heterogéneo y propietario se entienda con el microkernel.

	Linux	Windows
Saltos de privilegio para ejecutar	**1** (usuario → kernel)	**2** (usuario → ejecutivo → kernel)
Eficiencia	Mayor	Menor (cada salto cambia registros y modo)

> **Pantalla azul**
>
> Los drivers de terceros corren a nivel kernel sobre la HAL; si uno mal programado le manda una instrucción inconsistente, el sistema no puede aislar el error y colapsa (**BSOD**). Por eso Linux domina en embebidos e industria.

3.1.2 Propósito del sistema operativo y multitarea

- El SO administra los recursos (procesador, memoria, periféricos) y da un acceso **multiplexado** a cada uno: "el programa de programas".
- **Multitarea (material):** ilusión de ejecutar varios procesos a la vez. El planificador interrumpe el proceso, guarda su contexto (registros, PC, pila) y carga el siguiente; es **concurrencia multiplexada en el tiempo**, apoyada en el system tick (timers de la Unidad 2).

3.1.3 API y llamadas a sistema

Ningún programa toca periféricos o RAM sin pasar por el kernel ("la ventanilla del banco").

- **Consola:** `type`, `cat`, `sort`, `echo` piden al kernel E/S o manejo de datos.
- **Programación:** `printf`, `malloc`, `free`, `mkdir` (stdio.h, stdlib.h) son **envolturas** (wrappers) sobre las verdaderas llamadas al sistema.

3.1.4 Streams, redirección y tuberías

Descriptor	Flujo
0	STDIN (entrada)
1	STDOUT (salida de datos válidos)
2	STDERR (errores)

- `printf("texto")` es alias de `fprintf(stdout, "texto")`; la salida puede ir a un puerto serie o a un archivo, no siempre a una pantalla.
- `/dev/null` es "el vacío infinito": `1> /dev/null 2>&1` descarta salida y errores.
- **Tubería** `cat archivo.txt | grep "bus"`: el kernel conecta el STDOUT del primero con el STDIN del segundo **en RAM**, sin tocar disco.

3.1.5 Los 6 pasos de una llamada a sistema ("pregunta de examen")

1. **Enmascaramiento:** al compilar, la llamada (`fprintf`, `malloc`) se enmascara; el **linker** la resuelve contra los `.o` y las bibliotecas dinámicas (`.so`, glibc).
2. **Identificador único retrocompatible** de 128 bits (64 de versión de biblioteca + 64 de la llamada).
3. **Trampa y pila:** la llamada y sus argumentos se empujan (PUSH) a la pila y se activa la trampa.
4. **Cambio a modo kernel.**
5. **Desentrampar y ejecutar:** el kernel lee el identificador y los argumentos y ejecuta con privilegios totales.
6. **Retorno:** resultados en pila o registros, se desinstala la trampa y se regresa al espacio de usuario.

> **Ejemplo: malloc → sys_brk (material)**
>
> Si los fast bins están llenos, se invoca `sys_brk`: **EAX = 0x2D (45)** y se dispara **INT 0x80** (Linux x86); se ejecuta en el kernel para mover el límite del heap. Con memoria fragmentada, el kernel puede mover el proceso o mandar páginas al swap.
>
> Error común: creer que el compilador resuelve las llamadas; es el linker.

3.2.1 Programa, programa en ejecución y proceso

Concepto	Definición
Programa en disco	Secuencia pasiva, estática e inmodificable en medio no volátil, con direcciones **base cero**
Programa en ejecución	Secuencia activa y dinámica, **relocalizada** en RAM por el loader (difiere ≈ 20 % de los bytes del disco)
Proceso	La estructura completa que contiene al programa en ejecución: encabezado (PID, privilegios, estado), PC, pila, copia de registros y sección de datos (globales, heap)

"Si no saben la diferencia entre programa, programación y proceso, no merecen estar haciendo nada en esta carrera." Lo que se ejecuta y planifica es el **proceso**.

3.2.2 Unidades de ejecución en GPOS y RTOS

GPOS (Linux, Windows, macOS)	RTOS (FreeRTOS en ESP32, RP2040, Cortex-M)
Procesos e hilos; el **hilo** es la unidad mínima; todo proceso nace con un hilo principal; jerarquía padre-hijo	La unidad mínima es la **tarea**, con plazo estricto (deadline)
En Linux todo lo ejecutable es un proceso con atributos	Sin procesos aislados ni hilos tradicionales
Loader dinámico, bibliotecas `.so`	**Sin loader dinámico**: el linker genera el binario con todas las tareas y direcciones fijas
Memoria aislada por proceso	Un **superproceso raíz**; todas las tareas comparten memoria

Avisos

- Desde las Prácticas 5 y 8, la Raspberry Pi en Linux **modo consola** (no acepta interfaz gráfica). Archivos con número de lista a dos dígitos, enviados con 3 días de anticipación. Primer parcial: miércoles 14 oct.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Windows vs. Linux. Linux 1 salto de privilegio; Windows 2 (ejecutivo + kernel) y la HAL. Driver malo sobre la HAL = pantalla azul.

> **Llamada a sistema (examen)**
>
> Enmascarar (linker) → identificador de 128 bits → trampa y pila → modo kernel → ejecutar → regresar. `malloc` → `sys_brk`: EAX = 0x2D, INT 0x80.

Concepto	Para recordar
Streams	STDIN 0, STDOUT 1, STDERR 2; `printf` = `fprintf(stdout, …)`
Tubería `|`	Conecta procesos en RAM
`/dev/null`	Descarta
Programa	Estático, base cero
Programa en ejecución	Relocalizado en RAM
Proceso	Estructura que lo contiene (PID, PC, pila, registros, datos)
GPOS	Procesos e hilos, loader dinámico
RTOS	Tareas preenlazadas en un superproceso, memoria compartida, deadline